

# LightGBM

2026-04-17

## Introduction

LightGBM (Ke et al., 2017) is a gradient-boosting framework engineered for very large datasets. It introduces several optimisations over XGBoost: histogram-based split finding (substantially faster than sorted-value enumeration), leaf-wise tree growth (deeper trees per leaf count than level-wise), Gradient-based One-Side Sampling (GOSS) that prioritises high-gradient instances during training, and Exclusive Feature Bundling (EFB) that combines mutually exclusive sparse features. The result is typically 3–5× faster than XGBoost with comparable or slightly better accuracy on large tabular data.

## Prerequisites

A working understanding of gradient boosting, XGBoost or similar boosted-tree workflow, and the bias-variance trade-off in tree depth and leaf count.

## Theory

Three core innovations distinguish LightGBM from earlier gradient-boosting libraries:

- **Histogram binning:** pre-compute feature-value bins (default 255); split finding iterates over bins rather than sorted values, giving substantial speed and memory gains.
- **Leaf-wise growth:** at each iteration, expand the leaf with maximum loss reduction. Produces deeper, asymmetric trees given a fixed leaf count compared to level-wise growth, but at a higher over-fit risk.
- **GOSS:** sample instances by gradient magnitude, keeping the top- $a$  fraction with largest gradients and a random sample of the rest. The classifier focuses on the hard examples that contribute most to the gradient.

## Assumptions

Sufficient data for histogram bins to be informative; regularisation (`max_depth`, `num_leaves`, `min_data_in_leaf`) is tuned to prevent the over-fit risk inherent in leaf-wise growth.

## R Implementation

```
library(lightgbm)

set.seed(2026)
n <- 600
X <- matrix(rnorm(n * 5), n, 5)
y <- as.numeric(plogis(X[, 1] + 0.5 * X[, 2]^2) > runif(n))

idx <- sample(n, n * 0.8)
d_tr <- lgb.Dataset(X[idx, ], label = y[idx])
d_te <- lgb.Dataset.create.valid(d_tr, X[-idx, ], label = y[-idx])

params <- list(objective = "binary", metric = "binary_logloss",
               learning_rate = 0.05, num_leaves = 31,
```

```

        feature_fraction = 0.8, bagging_fraction = 0.8,
        bagging_freq = 5, min_data_in_leaf = 20)

model <- lgb.train(params, d_tr,
                  nrounds = 500, valids = list(val = d_te),
                  early_stopping_rounds = 20, verbose = -1)

cat("Best iter:", model$best_iter,
    "    best logloss:", round(model$best_score, 4), "\n")

imp <- lgb.importance(model, percentage = TRUE)
head(imp)

```

## Output & Results

`lgb.train()` returns a fitted model with the early-stopping iteration, best validation metric, and tree structure. `lgb.importance()` ranks features by gain (or split count, or cover) with optional percentage display.

## Interpretation

A reporting sentence: “LightGBM with `learning_rate` = 0.05, `num_leaves` = 31, feature and bagging fractions 0.8 converged at 184 boosting rounds with early stopping (validation log-loss 0.39); feature 1 contributed 53 % of total gain, confirming it as the dominant predictor in line with the simulated data-generating process. Training was approximately 4× faster than XGBoost on the same data.” Always state hyperparameters and early-stopping criteria.

## Practical Tips

- Constrain `num_leaves` (default 31) or `max_depth` to prevent the over-fit risk of leaf-wise growth; deeper trees are more accurate per round but more prone to over-fitting.
- LightGBM is typically 3–5× faster than XGBoost for the same number of trees; the speed gap widens further for > 1M rows.
- For categorical features, pass integer-encoded factors plus `categorical_feature` argument; LightGBM uses split-aware categorical handling that often outperforms one-hot encoding.
- A `learning_rate` of 0.01–0.05 is a sensible starting range; smaller values need more rounds but generalise better, larger values train faster.
- GPU training (`device_type` = "gpu") provides a drop-in 3–10× acceleration for very large data; CPU performance is already excellent.
- Compare LightGBM and XGBoost on the same data; results are usually very close, but the better model varies by task.

## R Packages Used

`lightgbm` for the canonical R interface; `xgboost` for the closely-related alternative; `catboost` for the third major gradient-boosting library (with native categorical handling); `parsnip::boost_tree(engine = "lightgbm")` for tidymodels integration; `bonsai` for the tidymodels-LightGBM bridge.

## For Reviewers

What to look for in a paper using this method.

- Common misapplications.
- Diagnostics that should be reported but often aren’t.

- **Red flags in tables and figures.**
- **What to verify.**
- **What an adequate Methods paragraph must contain.**

**See also — labs in this chapter**

- Cross-validation, nested CV, bootstrap .632+
- Regularisation: ridge, lasso, elastic net
- Trees, random forests, gradient boosting
- Interpretability and SHAP
- Tabular neural networks with torch
- Imaging and sequence models intro
- tidymodels pipelines
- FDR, knockoffs, replication crisis